

■■ Complete Spring Boot Parallel Pipeline Guide

Step-by-Step Implementation: ThreadPool Beans, RestClient Beans, Async Execution & AI RAG Integration

For GeM ML Microservice (Python) + Node AI RAG (pgvector/Gemini)

1. Parallel Pipeline Concept: Kya Hota Hai Aur Kyun Chahiye?

Jab user 1 PDF upload karta hai, toh hume do alag microservices ko simultaneously call karna hota hai:

- **Python ML Service** (<http://20.40.44.184>): OCR, Statutory Check, Authenticity, Forgery detection.
- **Node AI RAG Service** (<https://sih2026-86kl.onrender.com>): Text Chunking, Gemini Embeddings, pgvector storage.

Agar sequential karenge (ek ke baad ek), toh user ko 7+ seconds wait karna padega. **Parallel Pipeline** me Spring Boot ThreadPool use karta hai aur dono ko **ek hi waqt par 2 alag threads me run karta hai**. Total execution time sirf **~3 seconds** lagta hai!

2. Complete 5-Step Code Blueprint

Spring Boot me isko implement karne ke liye 4 main components lagte hain:

STEP 1: @EnableAsync + ThreadPoolTaskExecutor Bean (Multithreading setup).

STEP 2: RestClient Beans (ML Service aur Node AI RAG Service ke HTTP clients).

STEP 3: NodeRagServiceClient (Node AI RAG service ko call karne wala client).

STEP 4: ParallelDocumentService (Jahan CompletableFuture.supplyAsync se dono run hote hain).

STEP 5: Controller integration (User ko single consolidated JSON response dena).

STEP 1: Async ThreadPool Configuration Bean

Ye configuration Spring Boot ko batata hai ki background me kitne worker threads banakar parallel task run karne hain:

```
package com.example.Tender.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;
import java.util.concurrent.Executor;

@Configuration
@EnableAsync // Enables @Async and CompletableFuture background tasks
public class AsyncConfig {

    @Bean(name = "documentProcessingExecutor")
    public Executor documentProcessingExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(10);           // 10 worker threads active at all times
        executor.setMaxPoolSize(25);           // Max 25 concurrent tasks under heavy traffic
        executor.setQueueCapacity(100);        // Queue size before rejecting requests
        executor.setThreadNamePrefix("DocPipeline-"); // Useful in logs: [DocPipeline-1]
        executor.initialize();
        return executor;
    }
}
```

STEP 2: HTTP RestClient Beans Configuration

Python ML aur Node AI RAG dono services ke liye dedicated RestClient beans configure karte hain:

```
package com.example.Tender.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.client.SimpleClientHttpRequestFactory;
import org.springframework.web.client.RestClient;

@Configuration
public class ClientConfig {

    @Value("${ml.service.base-url:http://20.40.44.184}")
    private String mlServiceUrl;

    @Value("${ai.rag.service.base-url:https://sih2026-86kl.onrender.com}")
    private String nodeRagServiceUrl;

    @Bean
    public RestClient mlServiceRestClient() {
        SimpleClientHttpRequestFactory factory = new SimpleClientHttpRequestFactory();
        factory.setConnectTimeout(10000); // 10s connect timeout
        factory.setReadTimeout(60000);    // 60s read timeout (heavy ML OCR)
        return RestClient.builder().baseUrl(mlServiceUrl).requestFactory(factory).build();
    }

    @Bean
    public RestClient nodeRagRestClient() {
        SimpleClientHttpRequestFactory factory = new SimpleClientHttpRequestFactory();
        factory.setConnectTimeout(10000);
        factory.setReadTimeout(60000);
        return RestClient.builder().baseUrl(nodeRagServiceUrl).requestFactory(factory).build();
    }
}
```

STEP 3: Node AI RAG Service Client Implementation

Ye client Node AI RAG service ke endpoints (/api/ai/tender/process, /api/ai/bidder/process, /api/ai/bidder-tender-chat/ask) ko trigger karta hai:

```
package com.example.Tender.officer.client;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.MediaType;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;
import java.util.Map;

@Component
@RequiredArgsConstructor
@Slf4j
public class NodeRagServiceClient {

    private final RestClient nodeRagRestClient;

    // Trigger Node AI RAG service to process Tender PDF into pgvector embeddings
    public Map<String, Object> processTender(String tenderId) {
        log.info("Sending Tender ID {} to Node AI RAG service", tenderId);
        return nodeRagRestClient.post()
            .uri("/api/ai/tender/process")
            .contentType(MediaType.APPLICATION_JSON)
            .body(Map.of("tenderId", tenderId))
            .retrieve()
            .body(Map.class);
    }

    // Trigger Node AI RAG service for Bidder document
    public Map<String, Object> processBidderDocument(String tenderId, String bidderId, String docId, String docType) {

```

```
log.info("Sending Bidder {} Doc {} to Node AI RAG service", bidderId, docId);
return nodeRagRestClient.post()
    .uri("/api/ai/bidder/process")
    .contentType(MediaType.APPLICATION_JSON)
    .body(Map.of(
        "tenderId", tenderId,
        "bidderId", bidderId,
        "documentId", docId,
        "documentType", docType
    ))
    .retrieve()
    .body(Map.class);
}
```

STEP 4: Parallel Orchestrator Service (The Core Engine)

Ye main service hai jo CompletableFuture.supplyAsync use karke ML aur RAG ko parallel dispatch karti hai:

```
package com.example.Tender.officer.service;

import com.example.Tender.officer.client.MlServiceClient;
import com.example.Tender.officer.client.NodeRagServiceClient;
import com.example.Tender.officer.dto.ml.DocumentProcessResponse;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;
import java.util.Map;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.Executor;

@Service
@RequiredArgsConstructor
@Slf4j
public class ParallelPipelineService {

    private final MlServiceClient mlServiceClient;
    private final NodeRagServiceClient nodeRagServiceClient;
    private final CloudinaryService cloudinaryService;

    @Qualifier("documentProcessingExecutor")
    private final Executor documentProcessingExecutor;

    public Map<String, Object> processTenderInParallel(MultipartFile file, Long tenderId, String docType) {
        log.info("Starting Parallel Pipeline for Tender ID: {}", tenderId);

        // 1. First upload to Cloudinary CDN
        String cloudinaryUrl = cloudinaryService.uploadFile(file);

        // 2. Launch ML analysis in parallel Thread A
        CompletableFuture<DocumentProcessResponse> mlFuture = CompletableFuture.supplyAsync(() -> {
            log.info("[Thread A] Running Python ML Analysis...");
            return mlServiceClient.processDocument(file, docType, true);
        }, documentProcessingExecutor).exceptionally(ex -> {
            log.error("[Thread A] ML Service failed: {}", ex.getMessage());
            return null; // Fallback: Don't fail entire request if ML has glitch
        });

        // 3. Launch Node AI RAG Vectorization in parallel Thread B
        CompletableFuture<Map<String, Object>> ragFuture = CompletableFuture.supplyAsync(() -> {
            log.info("[Thread B] Running Node AI RAG Vectorization...");
            return nodeRagServiceClient.processTender(String.valueOf(tenderId));
        }, documentProcessingExecutor).exceptionally(ex -> {
            log.error("[Thread B] Node RAG Service failed: {}", ex.getMessage());
            return Map.of("status", "error", "message", ex.getMessage());
        });

        // 4. Wait for both parallel threads to complete (Non-blocking wait)
        CompletableFuture.allOf(mlFuture, ragFuture).join();

        // 5. Combine ML Results + RAG Vectorization Status
        DocumentProcessResponse mlResult = mlFuture.join();
        Map<String, Object> ragResult = ragFuture.join();

        return Map.of(
            "status", "SUCCESS",
            "tenderId", tenderId,
            "cloudinaryUrl", cloudinaryUrl,
            "mlAnalysis", mlResult != null ? mlResult : "ML Processing Failed",
            "ragVectorization", ragResult
        );
    }
}
```

■ **Why this is bulletproof:**

1. **Zero Lag:** Python ML and Node RAG run at the same time.
2. **.exceptionally() Resilience:** Agar ML me koi transient issue aaye, toh RAG continue rehta hai aur vice-versa.
3. **Dedicated ThreadPool:** Main HTTP request threads block nahi hote, jisse application super scalable rehti hai!